AI2 Assignment 1 Report - D4-V2 Search and Rescue

Final version to check: `codes/D4-V2_search_and_rescue_definitive`

Core theme: Search and Rescue - Unknown Victim Location

Planner used for final validation: ENHSP local Docker

1. Project objective and required scope

The project models a rescue robot operating in a damaged building. The topology of the building is known, but the victim location is not assumed in the exploration instance. The robot must move through the environment, explicitly inspect rooms, detect the victim, and perform rescue.

The required core deliverable is `codes/D4-V2_search_and_rescue`. It contains Q1 with classical PDDL and Q2 with PDDL+ health degradation. The optional variant interprets rescue as evacuation to a safe base. The final folder `codes/D4-V2_search_and_rescue_definitive` is the version to check because it combines transport, health degradation, patient assessment, autonomous branch selection, stabilization, and a mission deadline.

The central modelling issue is information acquisition: rescue must depend on detection, and detection must depend on inspection. The domain therefore separates movement, inspection, detection, rescue, and time-dependent failure instead of making the victim directly available from the initial state.

2. Planning abstraction

The building is represented as a discrete topological graph. Rooms are objects, and connectivity is encoded through explicit predicates. The robot does not reason about geometry, collision avoidance, mapping, localization, or low-level control. This is intentional: the PDDL model describes the high-level task structure, not the physical execution layer.

The classical part follows the standard domain/problem/plan split. The domain describes the general rules of the world: object types, predicates, and actions. Each problem file provides the concrete objects, initial state, and goal. The planner output is a sequence of actions that changes the initial state into a goal-satisfying state.

- one robot only, named `rescuebot`;
- known topology represented by `connected` facts;
- unknown victim location approximated through explicit inspection requirements;
- rescue blocked until the victim has been detected;
- time and health absent from Q1 but explicit in Q2 and the definitive extension;
- PDDL+ processes and events used where the world evolves without planner actions.

This abstraction is useful because it makes the rescue problem computable by a planner, but it also creates limitations. Classical PDDL does not contain real partial observability or probabilistic sensing; the model simulates the operational effect of sensing through symbolic predicates.

3. Repository organization

Folder	Role	Main content
codes/D4-V2_search_and_rescue	Core assignment deliverable	Q1 Basic PDDL, Q2 PDDL+, notes, saved plans
codes/D4-V2_search_and_rescue_variant	Optional variant	Rescue interpreted as transport to base
codes/D4-V2_search_and_rescue_definitive	Final version to check	Autonomous health-based branch selection, stabilization, deadline
Report/	Report deliverable	report.md and report.pdf
slide/	Presentation deliverable	Final presentation material

The top-level README explicitly states that the final version to check is `codes/D4-V2_search_and_rescue_definitive`, while preserving the core Q1/Q2 deliverable as the required assignment solution.

4. Q1 - Classical PDDL model

Q1 uses classical PDDL to model the search and rescue task without continuous time. The robot can move, inspect rooms, detect the victim if the inspected room contains the victim, and rescue only after detection.

The relevant folder is:

```
codes/D4-V2_search_and_rescue/Q1_basic_pddl/
```

File	Purpose
domain.pddl	Classical PDDL domain with movement, inspection, detection, and rescue

File	Purpose
problem_known_location.pddl	Instance where the victim location is effectively known through the model setup
problem_exploration.pddl	Instance requiring room exploration before rescue
plan_known_location.txt	Saved valid plan for the known-location case
plan_exploration.txt	Saved valid plan for the exploration case

Action	Meaning	Key modelling role
move	Move between connected rooms	Encodes navigation over the topology graph
inspect-empty-room	Inspect a room without finding the victim	Makes exploration explicit
inspect-victim-room	Inspect the victim room and produce detection	Connects sensing to later rescue
rescue	Complete rescue after detection	Prevents invalid rescue before information acquisition

The known-location plan moves from `entrance` to `room1`, inspects the victim room, and rescues. The exploration plan is longer: the robot moves through multiple rooms, performs inspections, detects the victim in the infirmary, and only then rescues.

Classical PDDL is deterministic and fully observable once the initial state is defined. For this reason, Q1 is an approximation of unknown victim location, not a true partial-observability solution. The model does not maintain a belief state; it forces inspection actions before rescue becomes applicable.

5. Q2 - PDDL+ model with health degradation

Q2 extends the domain with PDDL+ constructs. The motivation is that rescue missions are time-critical: even if the robot is idle or moving, the victim's condition may deteriorate. This cannot be represented adequately as a purely classical sequence of instantaneous symbolic actions.

The relevant folder is:

```
codes/D4-V2_search_and_rescue/Q2_pddl_plus/
```

File	Purpose
domain_plus.pddl	PDDL+ domain with numeric health, processes, and events
problem_fast_rescue.pddl	Instance where rescue is completed before health reaches zero
problem_delayed_rescue.pddl	Instance showing that delay can make rescue fail
plan_fast_rescue.txt	Saved output for the successful fast rescue
plan_delayed_rescue.txt	Saved delayed-output analysis and validation note

Construct	Purpose
victim-health	Numeric fluent representing the victim's remaining health
health-degradation	Process decreasing health continuously while the mission is active and the victim is alive
victim-dies	Event triggered when health reaches zero
finish events	Events that complete activities such as movement, inspection, rescue, or waiting

The fast rescue case starts with enough health and reaches the rescued state before health becomes zero. The delayed rescue case shows the interaction between sensing and time: the forced delay makes the mission fail at the model level because health reaches zero before rescue can be completed.

Some planner outputs returned an empty plan for delayed PDDL+ instances even though the goal was not true in the initial state. These outputs are preserved as planner compatibility observations, but they are explicitly marked as semantically unsupported in the corresponding plan file.

6. Optional transport-to-base variant

The optional variant changes the meaning of rescue. Instead of treating rescue as an on-site action, it models rescue as evacuation: after detecting the patient, the robot must load the patient, carry the patient through the graph, return to the base, and unload the patient there.

The relevant folder is:

```
codes/D4-V2_search_and_rescue_variant/
```

Action	Meaning
start-load-patient	Load the detected patient onto the robot
start-move-with-patient	Move while carrying the patient
start-unload-patient-at-base	Unload the patient at the safe base and complete rescue

The final goal requires both a symbolic rescue condition and the physical patient-location condition at base. This makes the variant more demanding than the core Q2 model because rescue is not complete when the patient is found; the patient must be transported back safely before health reaches zero.

This variant is not the primary deliverable. It is included to show a richer interpretation of rescue while keeping the original Q1/Q2 folder intact.

7. Final definitive autonomous extension

The final definitive model is the version to check:

```
codes/D4-V2_search_and_rescue_definitive/
```

This model combines the transport interpretation with an autonomous decision point. After the patient is detected, the robot assesses the patient's condition. The planner then follows one of two branches according to the numeric value of `victim-health` at assessment time.

Branch	Condition and behaviour	Expected output
Direct transport	Health is high enough to transport without stabilization	Plan contains transport actions and no start-stabilize-patient
Stabilize then transport	Health is low but still recoverable	Plan contains start-stabilize-patient before loading and transport
Too late	Delay makes recovery impossible before timeout/death	Planner reports Problem unsolvable

The final model also introduces `mission-time`, `mission-deadline`, a `mission-clock` process, and a `mission-timeout` event. This prevents the planner from waiting artificially until health crosses a branch threshold and makes the autonomous choice reflect the real state of the mission.

File	Role
domain_definitive_plus.pddl	Final autonomous PDDL+ domain
problem_definitive_direct_transport.pddl	High-health case
problem_definitive_stabilize_then_transport.pddl	Low-but-recoverable case
problem_definitive_too_late.pddl	Failure case
plan_definitive_direct_transport.txt	Validated direct-transport output
plan_definitive_stabilize_then_transport.txt	Validated stabilization output
plan_definitive_too_late.txt	Validated unsolvable output

8. Validation summary

The project was validated incrementally. Each model version was tested before its plan files were treated as final. The final definitive outputs were validated using ENHSP local Docker.

Test	Expected result	Status
Q1 known-location problem	Move to victim room, inspect, rescue	Passed
Q1 exploration problem	Explore and inspect before rescue	Passed
Rescue before detection	Rescue requires detection	Passed
Q2 fast rescue	Victim survives and rescue completes	Passed
Q2 delayed rescue	Delay explains failure through health degradation	Documented
Variant fast transport	Patient is loaded, carried, unloaded at base	Passed
Variant delayed transport	Delay makes transport rescue fail	Documented
Definitive direct branch	Solved without stabilization	Passed
Definitive stabilization branch	Solved with stabilization	Passed
Definitive too-late branch	Reported as unsolvable	Passed

The most important validation result is the final definitive triplet: one direct transport plan, one stabilization plan, and one unsolvable too-late case. Together they show that the planner's behaviour changes according to the numeric state of the patient rather than a manually selected symbolic branch.

9. Planner compatibility and known issues

The final validation uses ENHSP local Docker. The local setup was adopted because remote planner/package discovery was unreliable during development. ENHSP was suitable for the final PDDL+ model because it supports the required numeric, process, and event constructs used in the project.

Some compatibility issues were observed in older delayed PDDL+ tests, where the planner returned empty plans for goals that were not initially true. These outputs are not hidden. They are preserved in the relevant `plan_*.txt` files with notes explaining why the

result is not semantically accepted. The final definitive too-late case is cleaner: the planner reports `Problem unsolvable`, which matches the intended model behaviour.

The project also distinguishes between planner-controlled actions and automatic PDDL+ events. Saved plan files list planner actions, while finish events and threshold events are described separately because they are executed by the model dynamics rather than selected as agent actions.

10. Limitations

The model is intentionally high-level. It does not solve real robotic perception, mapping, motion planning, manipulation, or medical triage. The following limitations remain:

- classical PDDL does not represent real partial observability;
- inspection is deterministic and symbolic;
- the model does not use probabilistic belief updates;
- room connectivity is topological, not geometric;
- movement feasibility is assumed once rooms are connected;
- patient stabilization is abstracted as a symbolic/numeric operation;
- health degradation is linear and simplified;
- low-level execution errors are outside the model.

These limitations are acceptable for the assignment because the objective is to model the planning problem and its temporal extension, not to implement a complete robotic rescue system. The final definitive extension still improves the original model by making the rescue strategy depend on numeric health and by preventing artificial waiting through a deadline.

11. How to inspect the submitted results

The recommended inspection order is:

```
codes/D4-V2_search_and_rescue/Q1_basic_pddl/plan_known_location.txt
codes/D4-V2_search_and_rescue/Q1_basic_pddl/plan_exploration.txt
codes/D4-V2_search_and_rescue/Q2_pddl_plus/plan_fast_rescue.txt
codes/D4-V2_search_and_rescue_definitive/plan_definitive_direct_transport.txt
codes/D4-V2_search_and_rescue_definitive/plan_definitive_stabilize_then_transport.txt
codes/D4-V2_search_and_rescue_definitive/plan_definitive_too_late.txt
```

The first two files show the classical PDDL approximation. The Q2 fast file shows the PDDL+ health degradation model in a successful case. The three definitive files show the final autonomous behaviour and are the most important outputs for assessment.

12. Conclusion

The submitted project satisfies the D4-V2 requirements and extends them with an autonomous rescue strategy. Q1 demonstrates explicit exploration and detection before rescue. Q2 adds time pressure through PDDL+ health degradation and failure events. The optional variant models evacuation to base. The final definitive model selects between direct transport and stabilization using numeric patient health, and it correctly reports the too-late case as unsolvable.

The final version to check is therefore:

```
codes/D4-V2_search_and_rescue_definitive
```